

ClientOpsCRM 2.0.0 Architecture and Data Model

System boundary

ClientOpsCRM deliberately separates two concerns. ClientOps Relay 1.2.1 remains the ingress and workflow subsystem: deterministic C11 lead policy, PostgreSQL transaction state/outbox, n8n automation, authenticated companion boundary, deployment and operational tooling. ClientOpsCRM is a compiled C++17 operator-side state machine backed by SQLite.

This gives the system two independent persistence domains with an explicit bridge instead of making the CRM depend on the automation runtime for ordinary local operation.

Executable architecture

clientops is the top-level launcher. CRM commands invoke `crm/build/clientops-crm`; if the binary is missing or older than the source, the launcher builds the release target first. The CRM process opens one SQLite application database, enables foreign keys, applies the expected schema, installs trigger-enforced invariants, records the actor context, and executes one command.

The core implementation is intentionally small enough to audit as one translation unit while still separating concerns internally into database RAIL, prepared statements, validation/normalization, schema/invariants, command handlers, rendering, import/export and diagnostics.

Persistent entities

Accounts

Organizations are durable entities with name, normalized domain identity, website, industry, location, owner and active/inactive state. A non-empty normalized domain is unique.

Contacts

People may be attached to an account. The record retains display email/phone and normalized counterparts. A non-empty normalized email is unique; normalized phone is indexed for duplicate signaling rather than made globally unique because shared switchboards/family/business numbers are legitimate.

Leads

A lead is pre-opportunity work. It stores Relay identity when available, identity/contact fields, source, raw message, valid JSON metadata, score, priority, owner and lifecycle state (new, qualified, disqualified, converted). Relay ticket identity is unique when non-empty.

Pipelines and stages

A pipeline owns ordered stages. Each stage has a position, probability and kind: open, won, or lost. Each pipeline can have at most one won terminal and one lost terminal. The built-in Default Sales pipeline uses Qualification 20%, Discovery 40%, Proposal 65%, Negotiation 85%, Won 100%, Lost 0%.

Deals

A deal is an opportunity attached to one pipeline/stage and optionally to an account, primary contact and source lead. Money is `amount_minor` INTEGER; currency is a three-letter code; expected close is a validated calendar date; probability is synchronized to the stage; status is open, won, or lost.

Database triggers reject any insert/update where `stage.pipeline` does not equal `deal.pipeline` or where `stage.kind` does not equal `deal.status`. A lost deal without a reason is rejected below the command layer as well.

Tasks, notes and custom values

These use a controlled polymorphic reference (`account|contact|lead|deal`, integer entity ID). The command layer verifies target existence; doctor independently detects orphan polymorphic references. Tasks have explicit lifecycle state. Notes are append-oriented context. Custom field keys use a restricted identifier grammar and one value per entity/key.

Activities and audit

Activities form the operator-facing timeline: business events such as creation, qualification, conversion, stage moves, completion and closure. Audit rows capture inserts/updates/deletes with actor and JSON before/after material where applicable. Audit rows have database triggers that abort UPDATE and DELETE, making the application audit history append-only even if a caller bypasses the CLI and writes SQL directly.

Transaction model

Normal SQLite writes are atomic. Multi-record lead conversion explicitly starts `BEGIN IMMEDIATE`, resolves/creates account and contact state, creates the deal, links the lead, records activity, and commits. Any exception causes rollback through the transaction guard. This is directly tested by retrying an invalid conversion and comparing persistent counts before and after.

`BEGIN IMMEDIATE` also acquires the write reservation at transaction start, reducing the chance that a conversion does significant read-side work only to fail later when upgrading to a writer.

Search model

`crm_fts` is an SQLite FTS5 virtual table. Insert/update/delete triggers maintain search material for accounts, contacts, leads, deals and notes. Initialization rebuilds the index from authoritative tables so `init` is idempotent with respect to search state. Search results return entity type, entity ID and indexed text.

Normalization and exact arithmetic

Email identity is lowercased. Account domains are canonicalized by removing protocol/common `www` prefix and path/query material before lowercasing. Phones reduce to digits with international `00` prefix normalization for duplicate comparison.

Monetary input is parsed as decimal text and converted to integer minor units with at most two fractional digits. No binary floating-point value represents stored money. Forecast weighting performs integer arithmetic against the stored 0..100 probability.

Integrity doctor

doctor checks:

1. SQLite integrity_check.
2. foreign_key_check.
3. exactly one won/lost terminal stage per pipeline.
4. deal status/pipeline/stage consistency.
5. converted-lead links.
6. orphan task/note/custom references.
7. presence of append-only audit guards.

It returns failure if any invariant is broken.

Relay bridge

crm/scripts/import-relay.sh reads the canonical `clientops.leads` table from the live Relay PostgreSQL container using PostgreSQL's own CSV emitter. It maps Relay UUID to `relay_ticket_id` and serializes policy/routing provenance to JSON. The data is streamed, not staged in an intermediate application file. The CRM's import path validates records and skips already-known Relay tickets when requested.

Failure semantics

The design prefers explicit rejection over guessing. Invalid email/date/money/priority/JSON input is rejected. Unknown schema versions are rejected. Duplicate unique identities are rejected. Illegal deal transitions are rejected. Audit mutation is rejected. Conversion from a non-qualified/already-converted lead is rejected. The command exits nonzero and does not manufacture a successful result.